

Trabalho Prático 03 – Hierarquia de Memória

Disciplina: CCF 252 – Organização de Computadores I

Alunos: Otávio Soares Pedroso – 6593, Pedro Mendes Gonzaga – 6563

1. Organização da Cache do Computador Utilizado

De acordo com as especificações da ferramenta [lscpu](#) cruzadas com as informações do site [CPU-World](#), os testes foram executados em um processador **13th Gen Intel(R) Core(TM) i7-13620H** (Raptor Lake-H). A arquitetura híbrida (6 Performance-cores e 4 Efficient-cores) possui a seguinte hierarquia de memória cache:

- **L1 de Dados (L1d):** Capacidade total de 416 KiB (dividida entre os núcleos). Possui blocos de 64 bytes e associatividade por conjunto de 12 vias (nos P-cores).
- **L1 de Instruções (L1i):** Capacidade total de 448 KiB. Possui blocos de 64 bytes e associatividade de 8 vias.
- **Cache L2:** Capacidade total de 9.5 MiB, com associatividade de 10 vias. Memória unificada para dados e instruções, alocada por núcleo/cluster.
- **Cache L3:** Capacidade total de 24 MiB, com associatividade de 12 vias. Memória unificada e compartilhada entre todos os núcleos (P-cores e E-cores) do processador.

2. Descrição dos Algoritmos de Ordenação

Para este trabalho, além do Bubblesort, Radixsort e Quicksort, optamos pelo **Insertionsort** para compor o quarto algoritmo exigido. As implementações em C utilizadas nos testes foram baseadas em materiais de apoio como o [GeeksforGeeks](#).

- **Bubblesort:** É um algoritmo simples de ordenação que compara pares de elementos adjacentes e os troca de lugar se estiverem na ordem errada. Esse processo é repetido até que a lista esteja ordenada. Devido às repetidas trocas, possui complexidade $O(n^2)$, sendo ineficiente para grandes conjuntos de dados.
- **Insertionsort (Algoritmo de Escolha):** Constrói a lista final ordenada um elemento de cada vez. Ele pega um elemento da parte não ordenada e o insere na posição correta na parte já ordenada. É muito eficiente para listas pequenas ou quase ordenadas, mas sua complexidade geral também é $O(n^2)$.
- **Quicksort:** Um algoritmo eficiente baseado no paradigma de divisão e conquista. Ele escolhe um elemento como pivô e particiona o vetor em dois subvetores: elementos menores que o pivô e

maiores que o pivô. Em seguida, ordena recursivamente as duas metades. Na média, sua complexidade é $O(n \log n)$.

- **Radixsort:** Um algoritmo de ordenação não comparativo que ordena inteiros processando os dígitos individuais de cada número, geralmente começando pelo dígito menos significativo (LSD). Ele utiliza algoritmos de ordenação estáveis como o Counting Sort como sub-rotina para cada dígito, possuindo uma complexidade de tempo linear $O(nk)$, onde k é o número de dígitos.

3 e 4. Desempenho e Comparação dos Algoritmos (Ferramenta `perf`)

Para a execução dos testes e comparação de resultados, desenvolvemos um código em C que gera vetores com valores aleatórios (utilizando uma semente fixa com `srand(42)` para garantir a reprodutibilidade dos dados nas repetições do experimento). As métricas apresentadas abaixo foram extraídas por meio da ferramenta `perf stat` englobando as tags solicitadas: `cache-references`, `cache-misses`, `task-clock`, `cycles` e `instructions`.

Entradas Pequenas (N = 1.000 elementos)

Algoritmo	task-clock (ms)	cycles	instructions	cache-references	cache-misses
Bubblesort	2.35	2,266,742	5,448,191	5,060	1,489
Insertionsort	0.93	780,851	2,208,154	4,401	2,098
Radixsort	1.33	483,898	406,022	5,170	3,425
Quicksort	1.10	455,860	353,135	4,627	1,794

Entradas Grandes (N = 10.000 elementos)

Algoritmo	task-clock (ms)	cycles	instructions	cache-references	cache-misses
Bubblesort	258.63	~552,267,402	~1,154,881,228	84,682	12,946
Insertionsort	18.58	26,415,157	202,494,751	6,091	2,545
Radixsort	2.25	1,849,982	2,804,427	4,778	2,473
Quicksort	3.20	3,283,115	3,448,545	16,365	6,798

Análise dos Resultados:

- Para **N = 1.000**, o tamanho reduzido do vetor (apenas 4 KB) cabe inteiramente na Cache L1, o que mantém o número de *cache-misses* muito baixo para todos os algoritmos. O gargalo fica no número de ciclos e instruções. O **Quicksort** e o **Radixsort** se destacaram com menos de 500 mil ciclos cada e poucas centenas de milhares de instruções.
- Para **N = 10.000**, as diferenças se tornam gritantes. O **Bubblesort** precisou de mais de 1 bilhão de instruções e demorou 258 ms, tornando-se impraticável. O **Insertionsort**, apesar de ser também $O(n^2)$, foi muito mais rápido (18 ms) por ter menor overhead de operações. O **Radixsort** foi o mais veloz em termos absolutos (2.25 ms) com apenas 2.8 milhões de instruções, seguido do **Quicksort** (3.20 ms), demonstrando a superioridade dos algoritmos $O(n \log n)$ e $O(nk)$ para entradas maiores.

5. Simulação com o Valgrind (Cachegrind)

A ferramenta `valgrind --tool=cachegrind` simulou a execução considerando uma arquitetura de cache, coletando eventos específicos da L1 de dados (D1) e do último nível de cache (LL).

Para entradas pequenas (N = 1.000):

Algoritmo	D1 miss rate	LLi misses	LL misses (Last Level)
Bubblesort	0.1%	1,189	2,657
Radixsort	1.6%	1,204	2,739
Quicksort	1.2%	1,199	2,666
Insertionsort	0.3%	1,187	2,655

Observação: Para uma entrada de 1000 elementos (4000 bytes em inteiros de 32-bits), todo o vetor cabe em uma única página de cache L1 de tamanho moderno (tipicamente de 48 KB). Como resultado, na configuração **Padrão**, o número de *cache misses* se referiu majoritariamente à inicialização e fallouts da rotina do sistema.

Testes alterando parâmetros da L1 de Dados (D1)

Para observar o simulador Valgrind na prática, realizamos novos testes com o **Bubblesort** forçando uma restrição rigorosa na Cache L1 de dados (apenas 1 KiB). O objetivo desta etapa foi analisar o impacto prático de uma falta proposital de espaço na localidade espacial:

Parâmetros D1 Simulados	Capacidade	Linha (Block)	Taxa de Miss (D1)
Padrão da Máquina	~48 KiB	64 bytes	0.1%
-D1=1024,2,64	1 KiB	64 bytes	1.6%
-D1=1024,2,32	1 KiB	32 bytes	2.8%

Conclusão da Simulação:

1. **Redução da Capacidade:** Ao reduzirmos a capacidade da cache para 1 KiB (`--D1=1024, 2, 64`), o vetor de 4000 bytes excedeu o limite físico no nível L1. Como resultado, observamos um aumento massivo de *capacity misses* durante o Bubblesort (mais de 44 mil erros), obrigando o processador a recorrer frequentemente ao último nível de cache (LL).
2. **Redução do Tamanho da Linha:** Em seguida, mantivemos a restrição de 1 KiB de capacidade total, mas reduzimos o tamanho do bloco (linha) pela metade (`--D1=1024, 2, 32`). O número de erros sofreu um aumento drástico indo para 70.391 *misses*. Isso ocorreu porque blocos menores conseguem carregar menos elementos adjacentes para a cache a cada leitura, prejudicando severamente a **localidade espacial** tão explorada nos laços de repetição dos algoritmos iterativos.

6. Otimização de Layout de Memória: AoS vs SoA (Array of Structs vs Struct of Arrays)

A fim de explorar diretamente os princípios de localidade espacial e a utilização de linhas de cache em contexto prático, decidimos focar na otimização do **Layout de Memória** de uma estrutura de dados de grande escala (simulando 10 milhões de "partículas" em um motor de física).

Otimização implementada:

Implementamos duas abordagens em linguagem C com o objetivo único de atualizar a posição `x` dessas partículas iterativamente (`x += vx`):

1. **AoS (Array of Structures - Abordagem Tradicional):** Na forma mais comum de programação estruturada, criamos uma `struct` contendo os atributos mistos da partícula (`x`, `y`, `z`, `vx`, `vy`, `vz`, massa, etc). Ao iterar sobre o vetor para ler a propriedade `x`, o processador carrega toda a `struct` para a linha de Cache L1 (64 bytes). O revés dessa abordagem é que a cache acaba sendo preenchida com dados desnecessários para o laço numérico atual (como massa e eixos não utilizados), o que desperdiça drasticamente o desempenho de leitura.
2. **SoA (Structure of Arrays - Abordagem Otimizada):** Nesta segunda estratégia, isolamos as propriedades em arrays independentes (um array dedicado apenas para as posições `x`, outro apenas para `vx`, etc.). Dessa forma, ao acessar `x[i]`, a arquitetura da linha de cache (64 bytes) carregada da RAM vem preenchida de carona com os próximos 16 números da posição `x` contíguos, garantindo aproveitamento máximo dos dados nas repetições consecutivas do laço `for`.

Comparação de Desempenho (N = 10.000.000):

Métricas extraídas utilizando a ferramenta `perf stat` com a flag `-O2`:

Métrica	AoS (Array de Structs)	SoA (Struct de Arrays)
---------	------------------------	------------------------

Métrica	AoS (Array de Structs)	SoA (Struct de Arrays)
Tempo Total (task-clock)	143.60 ms	26.36 ms
Instruções	~161 Milhões	~35 Milhões
Ciclos	~152 Milhões	~26 Milhões
Cache Misses	~5.46 Milhões	~741 Mil (Apenas 741 mil erros)

Conclusão:

A modificação na disposição dos dados transformou um loop que demorava 143 ms para um tempo ultrarrápido de **26 ms (Mais de 5x mais rápido!)**. Note como o número de *cache misses* na abordagem tradicional foi de 5.4 Milhões, enquanto a abordagem **SoA** diminuiu brutalmente para cerca de 740 mil *misses* ao simplesmente permitir que o processador lesse apenas os valores estritamente necessários x e vx contíguos por linha de cache. Este é um dos princípios mais poderosos utilizados no paradigma *Data-Oriented Design* da atualidade.